

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.12

Broadcasting & Vectorised Operations

Day 13 — why the loop disappears entirely

90 min

Concepts + live code

30 min

Practice notebook

2

Rules to remember

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Yesterday's last challenge — why did this work?

```
data = np.array([[10., 200., 3000.],
                 [20., 400., 1000.],
                 [30., 600., 2000.]])

col_min = data.min(axis=0)      # shape (3,)

scaled = (data - col_min) / (col_max - col_min)
```

data is (3, 3). col_min is (3,). Different shapes — and yet no error.

Four quick questions first

Is `arr[1:3]` a copy?

→ No — a view.

`m[:, 0]` — row or column?

→ Column.

Why `&` and not `and`?

→ `and` needs one `True/False`.

What does `axis=0` collapse?

→ The rows.

The answer has a name: broadcasting

NumPy stretched the 3-element row across all three rows of the array, without ever copying it. That is the mechanism behind every operation you did yesterday — `marks * 2`, `marks >= 40`, and the scaling above. Today we make the rule explicit, so you can predict when it will work and read the error when it does not.

How today's two hours are structured

1

Vectorisation

Element-wise operations and ufuncs

20 min

2

Broadcasting

The two rules, and reading the error

30 min

3

Aggregation

sum, mean, std — and axis again

20 min

4

Vectorised logic

where, clip, any, all

15 min

5

Mini build

Scaling a dataset two ways

5 min

6

Practice notebook

You code, I walk around

30 min

Section 2 is two rules and one error message. Learn to read that error and NumPy stops being mysterious.

By the end of today you will be able to

1

Vectorise

Replace a loop over numbers with a single expression

2

Predict

Say whether two shapes will combine — before you run it

3

Diagnose

Read a broadcast error and know which axis is wrong

4

Summarise

Aggregate along an axis, and scale a real dataset

Why this is the session that pays off later

Broadcasting is not a NumPy curiosity — it is how pandas subtracts a column mean from a DataFrame, how scikit-learn scales features, and how every neural network adds a bias vector to a batch of inputs. The shapes get bigger; the rule stays exactly the same.



Vectorisation

The loop disappears · 20 minutes

- Element-wise arithmetic
- Comparison and logic
- Universal functions

Every operator works element by element

```
a = np.array([1, 2, 3])
```

```
b = np.array([10, 20, 30])
```



Position by position. No loop, no index.

```
a + b    a - b    a * b    a / b    a ** 2    a % 2    and also: a > 2    a == b
```

⚠ a * b is NOT matrix multiplication

```
a * b    -> [10 40 90]    element-wise
a @ b    -> 140           dot product

# * multiplies matching positions
# @ is the matrix operator
```

You will need @ in Module 4. Until then, * is what you want.

Universal functions (ufuncs)

```
np.sqrt([1, 4, 9])    -> [1. 2. 3.]
np.exp    np.log    np.log10
np.sin    np.cos    np.abs
np.round(x, 2)    np.floor    np.ceil

# math.sqrt(arr) FAILS - it wants one number
```



Broadcasting

Two rules, one error message · 30 minutes

- The scalar case
- The two rules
- Reading the error
- Making a column

Broadcasting — the two rules

Line the shapes up from the RIGHT. Two dimensions are compatible when they are equal, or one of them is 1.

A shorter array is padded on the LEFT with 1s until both have the same number of dimensions. Wherever a dimension is 1, that array is stretched to match — without ever being copied in memory.

Worked examples — align right, then compare

(3, 4)	(3, 4)	(3, 4)	(3, 1)	(3, 4)
(4,)	(3,)	(3, 1)	(1, 4)	scalar
-----	-----	-----	-----	-----
(3, 4) OK	ERROR	(3, 4) OK	(3, 4) OK	(3, 4) OK
	4 vs 3			

Read the last column first: 4 vs 4 fine · 4 vs 3 clash · 4 vs 1 stretch

What “stretching” actually looks like

data (3, 3)

10	200	3000
20	400	1000
30	600	2000

col_min (3,)

10	200	1000
stretched down ↓		
10	200	1000
10	200	1000

result (3, 3)

0	0	2000
10	200	0
20	400	1000

The pale rows are not real. NumPy never builds them — it just reads the same three numbers again for each row.

This is why broadcasting is free

Subtracting a column mean from a million-row table does not create a million copies of the mean. NumPy walks the big array and re-reads the small one, so the memory cost is zero and the speed is the same as any other element-wise operation. Writing the loop yourself would be both slower and larger.

When it fails — and how to read the error

The error you will meet most often

```
a = np.ones((3, 4))
b = np.ones(3)
a + b

ValueError: operands could not be
broadcast together with shapes (3,4) (3,)
```

It tells you both shapes. Align them from the right: 4 against 3. Neither is 1, so they clash.

The fix: say which axis you meant

```
b.reshape(-1, 1)    shape (3, 1)
b[:, np.newaxis]    the same thing
b[:, None]          shorthand

a + b.reshape(-1, 1)
-> works: (3,4) + (3,1)
```

A (3,) is a ROW by default. Making it (3,1) declares it a column.

The three-step debug, every time

1 Print both `.shape` values

2 Align them from the right

3 Ask which one should be stretched

Nearly every shape error in NumPy, pandas and scikit-learn is solved by those three steps. The message always names both shapes — most of the work is just reading it carefully.

3

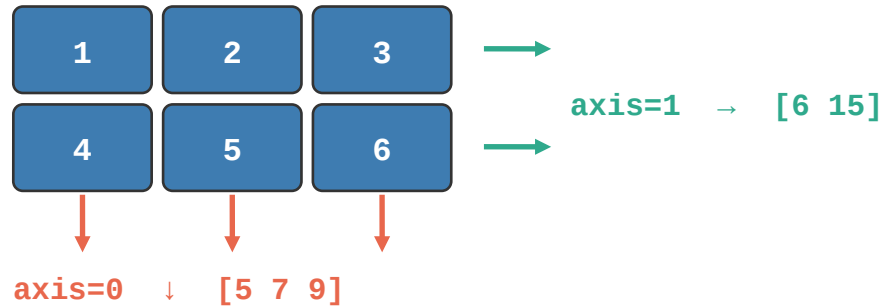
Aggregation

Collapsing an axis · 20 minutes

- sum, mean, min, max
- The axis rule
- argmin & argmax
- std and ddof

Aggregation — the axis you name disappears

```
m = np.array([[1, 2, 3], [4, 5, 6]])
```



```
m.sum() -> 21      no axis: everything, one number
```

The functions you will use

```
sum    mean    min    max    std    var  
argmin argmax    the POSITION, not the value  
cumsum cumprod    running totals
```

The rule that removes the confusion

```
m.shape            (2, 3)  
m.sum(axis=0).shape (3, )  
m.sum(axis=1).shape (2, )
```

The axis you NAME is the one that vanishes from the shape. axis=0 removes the rows, leaving one value per column. Forget “rows” and “columns” — look at what disappears from the shape and you will never get it backwards.

! std defaults to the POPULATION formula

```
arr.std()            divides by n  
arr.std(ddof=1)      divides by n-1
```

Module 2 will care about this difference.

4

Vectorised Logic

if/else without a loop · 15 minutes

- `np.where`
- `np.clip`
- `any` & `all`

Conditional logic, applied to a whole array

np.where — a vectorised if/else

```
marks = np.array([88, 35, 71, 22, 64])

np.where(marks >= 40, "pass", "fail")
-> ['pass' 'fail' 'pass' 'fail' 'pass']

np.where(condition, if_true, if_false)
```

Day 6's "A if cond else B", applied to every element. Like that one, nothing is dropped — the result is always the same length as the input.

np.clip — squeeze into a range

```
np.clip(marks, 40, 80)
-> [80 40 71 40 64]

# 88 came down to 80
# 35 and 22 came up to 40
# 71 and 64 were already inside
```

Useful for capping outliers — which is Day 20's topic.

any and all — collapsing a mask to one answer

```
(marks > 90).any()   -> False   is ANY above 90?
(marks > 20).all()   -> True    are ALL above 20?
(marks < 40).sum()   -> 2       how many fail?
```

These are what you use inside an if statement. A raw mask cannot go there — that is what yesterday's "truth value is ambiguous" error was telling you. `any()` and `all()` turn the whole array into the single True or False that if needs.

Debug corner — eight broadcasting bugs

You wrote	What happens	Fix
<code>(3,4) + (3,)</code>	ValueError — 4 against 3	<code>b.reshape(-1, 1)</code>
<code>a * b</code> for a dot product	element-wise, wrong answer	<code>a @ b</code>
<code>math.sqrt(arr)</code>	TypeError — wants one number	<code>np.sqrt(arr)</code>
<code>if marks > 40:</code>	ValueError: truth value ambiguous	<code>(marks > 40).any()</code>
<code>arr.mean(0)</code> meaning rows	gives one value per COLUMN	the named axis disappears
<code>argmax</code> expecting a value	it returns the POSITION	<code>arr[arr.argmax()]</code>
<code>arr.std()</code> as sample std	divides by n, not n-1	<code>arr.std(ddof=1)</code>
<code>np.where(cond, a)</code>	TypeError — needs three arguments	<code>np.where(cond, a, b)</code>

Row 1 is the one you will hit today. Print both shapes, align them from the right. All eight are in the practice notebook.

Putting it together — scaling a dataset two ways

```
import numpy as np

# 3 samples, 3 features - on wildly different scales
data = np.array([[10., 200., 3000.],
                 [20., 400., 1000.],
                 [30., 600., 2000.]])

# ---- 1. Min-max scaling: squeeze every column into 0-1
col_min = data.min(axis=0)      # (3,) one per column
col_max = data.max(axis=0)      # (3,)
minmax = (data - col_min) / (col_max - col_min)

# ---- 2. Standardisation: mean 0, standard deviation 1
col_mean = data.mean(axis=0)
col_std = data.std(axis=0)
z = (data - col_mean) / col_std

print("min-max:\n", minmax)
print("z-score:\n", z.round(3))

# proof it worked, without a single loop:
print("minmax range :", minmax.min(), minmax.max()) # 0.0 1.0
print("z means      :", z.mean(axis=0).round(10))  # [0. 0. 0.]
print("z stds       :", z.std(axis=0).round(10))    # [1. 1. 1.]
```

These two lines are the whole of feature scaling. In Module 3 scikit-learn does it for you — and now you know exactly what it is doing.



Broadcasting, twice

(3,3) minus (3,) in both formulas



axis=0

one value per column, as always



No loops at all

nine numbers, one expression



Min-max

fixed 0-1 range, keeps the shape



Z-score

mean 0, std 1, handles outliers better



Verified, not assumed

the last three lines prove it



You will meet these again

MinMaxScaler and StandardScaler

Practice session — 30 minutes



Predict the output

Shapes, broadcasting, axis, where

8 questions



Spot & fix the bug

Shape clash, dot product, ambiguous if

4 questions



Write the code

Scale, normalise, aggregate, label

5 tasks

Your notebook

[Day13_Practice_Questions.ipynb](#)

- Print both `.shape` values
- Align them from the right
- Raise a hand — I am walking around
- Solutions shared after class

Write no loops today. If you reach for `for i in range(len(arr))` , stop and ask what the whole-array version would be.

Recap

- 1 Every operator works element by element, with no loop.
- 2 $a * b$ multiplies positions; $a @ b$ is the dot product.
- 3 Use `np.sqrt`, not `math.sqrt` — `math` wants one number.
- 4 BROADCASTING: align the shapes from the RIGHT.
- 5 Two dimensions fit if equal, or if one of them is 1.
- 6 A shorter shape is padded on the LEFT with 1s.
- 7 Nothing is copied — the small array is simply re-read.
- 8 A (3,) is a row. Use `reshape(-1, 1)` to make a column.
- 9 On any shape error: print both shapes, align right.
- 10 The axis you name is the one that disappears.
- 11 `argmax` gives a POSITION; `std` defaults to `ddof=0`.
- 12 `np.where` is if/else; `any()` and `all()` feed an if.

Homework

Standardise a 10x4 random array and prove each column has mean 0 · Build a 10x10 multiplication table with broadcasting and no loops · Write down three shape pairs that will NOT broadcast, and why

Next class — Topic 1.13

NumPy statistical functions: mean, median, percentile, correlation — and handling missing values with nan-aware functions